



Politecnico
di Torino

Generative Artificial Intelligence for graphics and multimedia

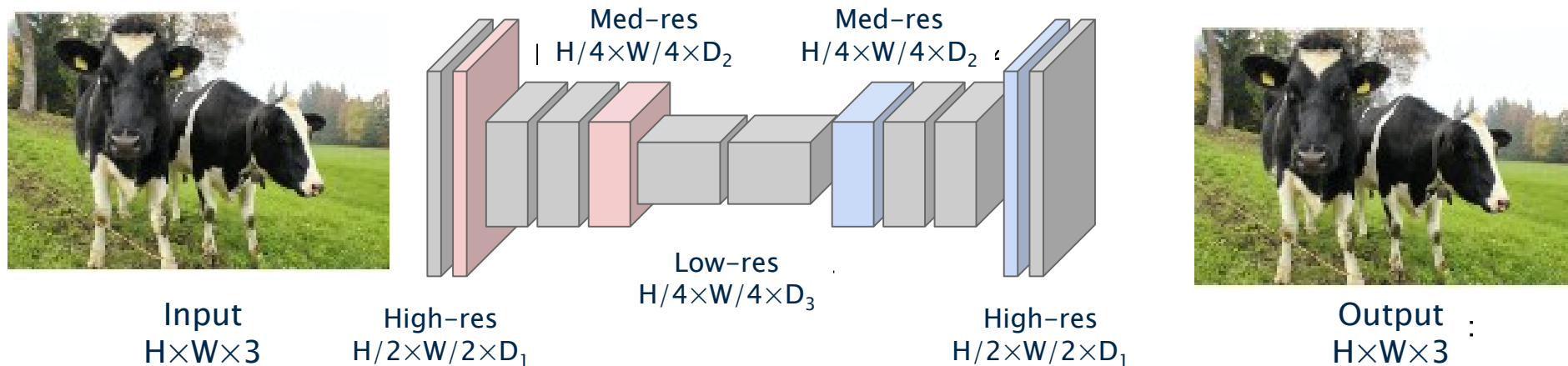
Variational autoencoders

Prof. Lia Morra



Recall: the encoder-decoder architecture

Design a network as a bunch of convolutional layers with downsampling and upsampling inside the network



After having reduced the spatial resolution in the first half of the network, it is increased again in the second part to match input size

In-network upsampling: Unpooling

Max pooling: remember which element was max

1	3	6	3
3	5	2	1
1	2	2	1
7	3	4	8

4×4



5	6
7	8

2×2

Rest of the network

...

1	2
3	4

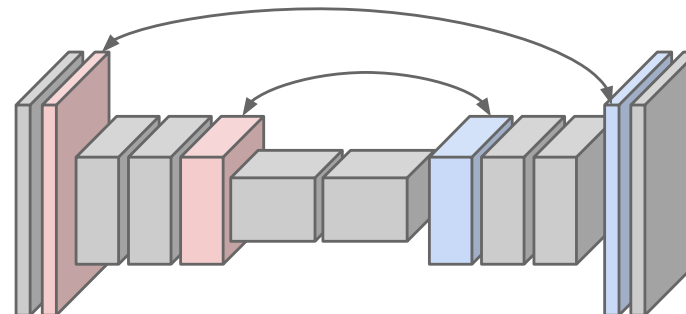
2×2

Max unpooling: use posit. from pooling layer

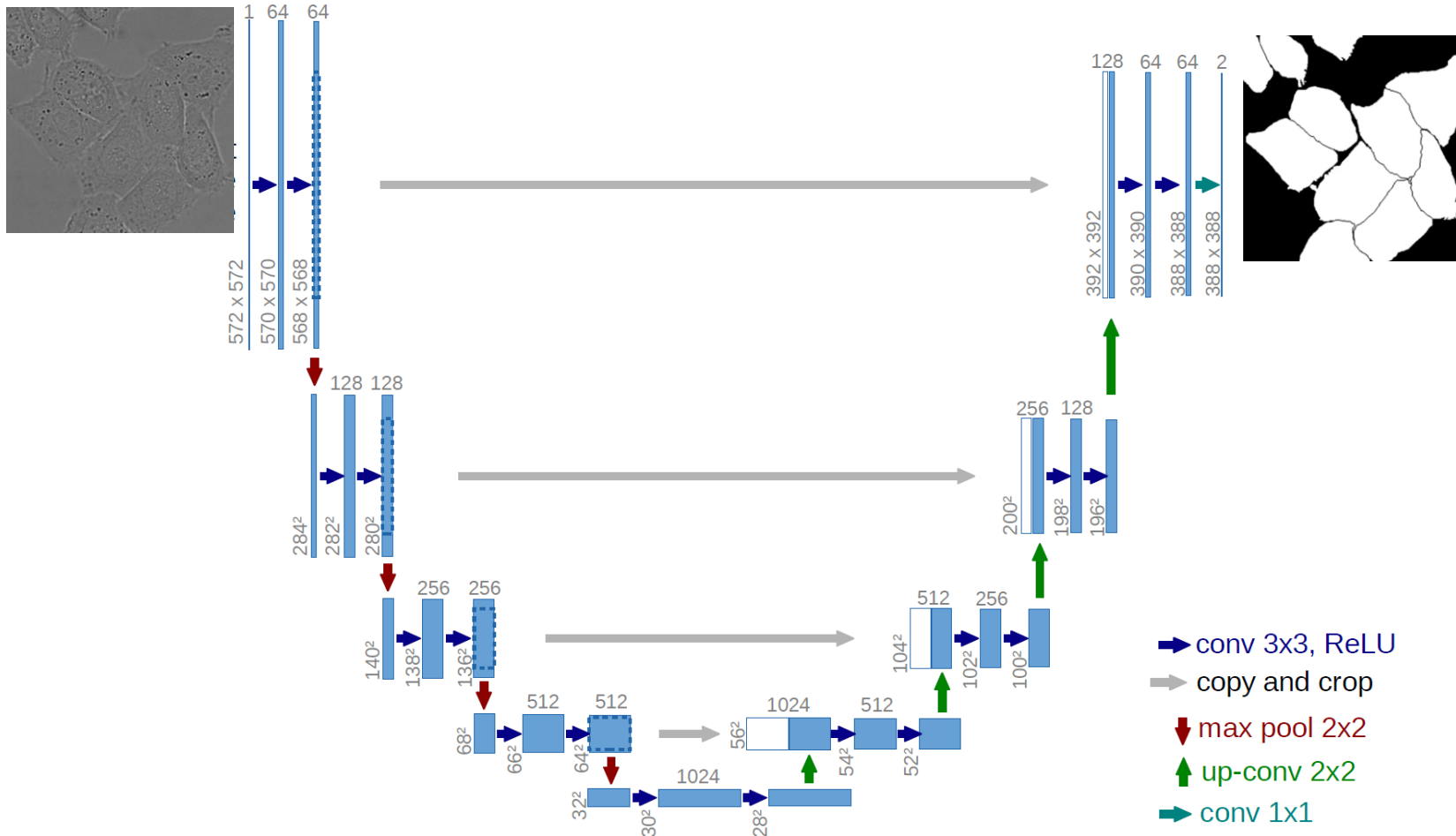
0	0	2	0
0	1	0	0
0	0	0	0
3	0	0	4

4×4

Corresponding pairs of downsampling and upsampling layers

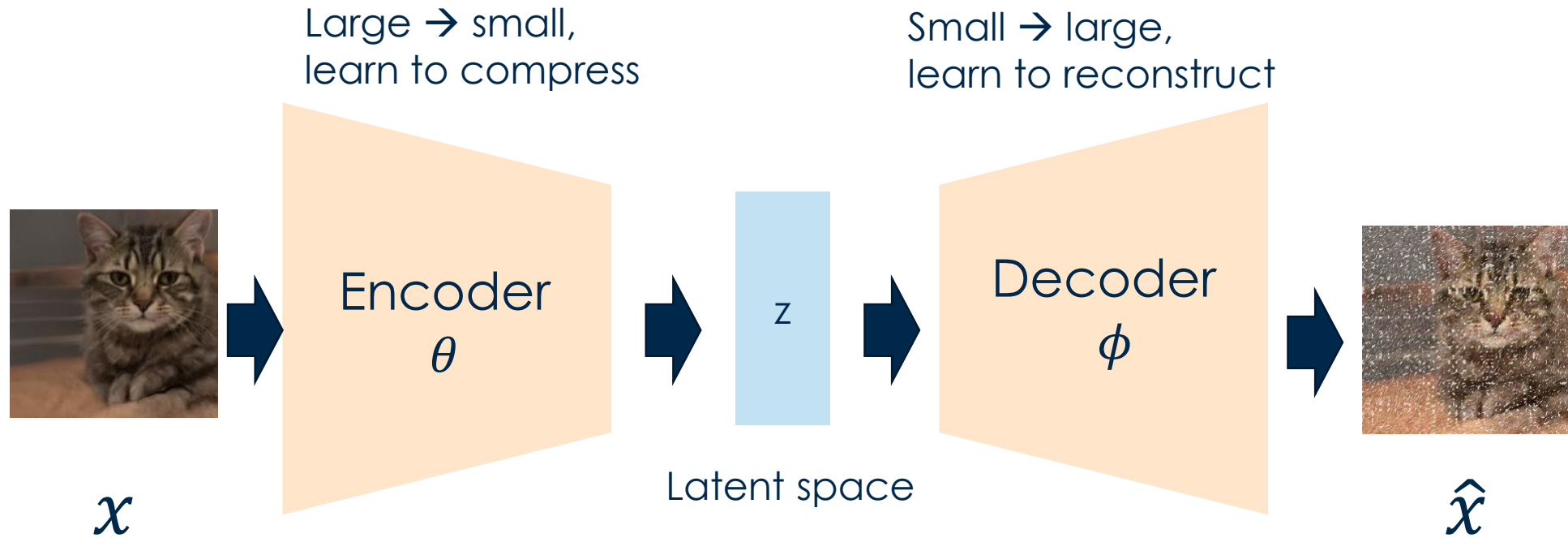


Architectural variants: U-Net



[Ronneberger O. et al. 2015, U-Net: Convolutional Networks for Biomedical Image Segmentation]

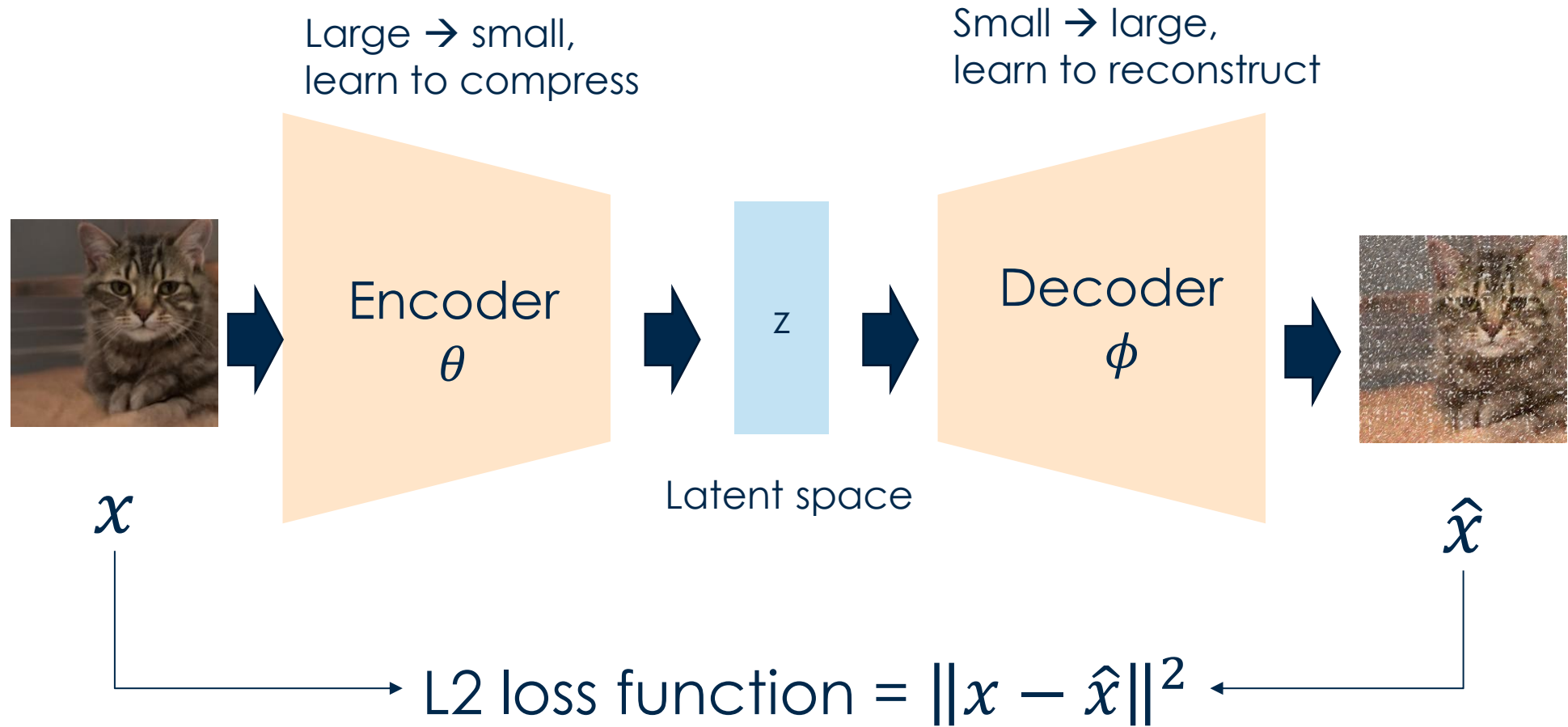
From encoder-decoder to auto-encoder



Some questions...

- What **loss function** is typically used to train an autoencoder for image reconstruction?
- What is the role of the **decoder** in a seq2seq Transformer vs. in an autoencoder — what do they have in common?
- Can you **sample** new images from a trained classic autoencoder by randomly picking a point in the latent space? Why or why not?

From encoder-decoder to auto-encoder



Loss function: alternatives

Mean Squared Error (MSE)

- measures the average squared pixel-wise difference between the input and its reconstruction.
- Assumes implicitly that pixel errors follow a Gaussian distribution.
- Tends to produce slightly blurry outputs because it penalizes large deviations heavily and favors averaging.

Binary Cross-Entropy (BCE)

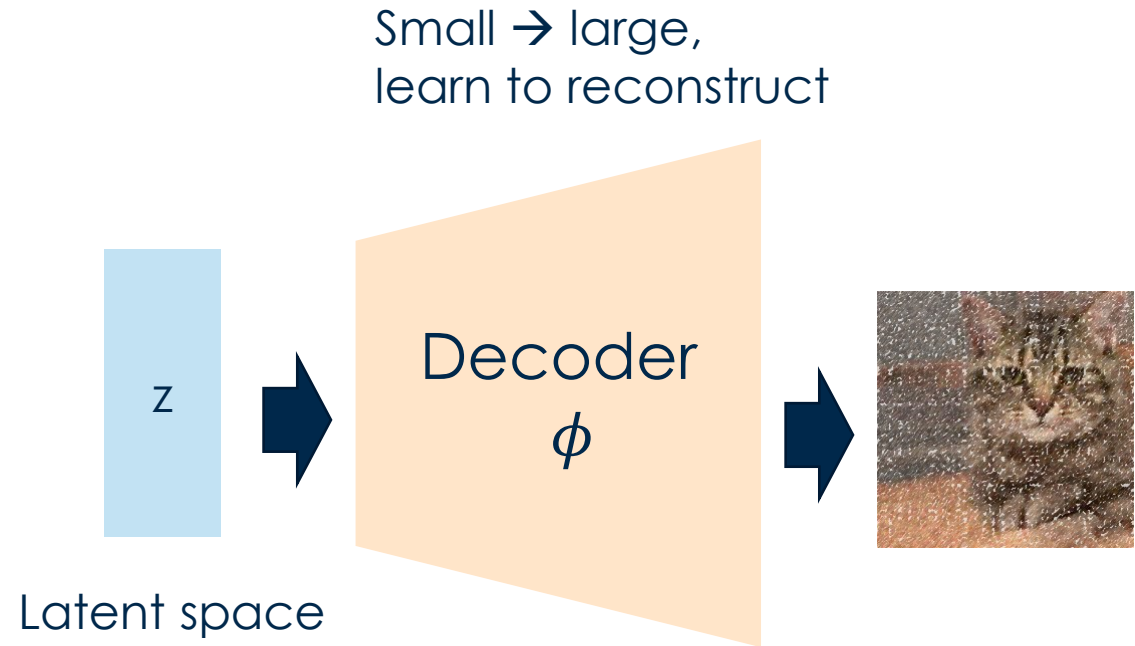
- Used when pixel values are normalized to $[0,1]$ and treated as Bernoulli probabilities.
- More appropriate for binary or near-binary images (e.g., MNIST).

Transformer decoder vs. VAE decoder

	seq2seq Transformer Decoder	Autoencoder Decoder
Input	Encoder hidden states + previously generated tokens	Latent code z
Output	Next token in a sequence	Reconstructed input $\hat{x}$
Mechanism	Cross-attention over encoder output + causal self-attention	Learned upsampling / transposed convolutions / MLP
Goal	Generate a target sequence conditioned on source	Recover the original input from its compressed representation

Sampling at test time

“Test time”:
→reconstruction from
‘random’ vector

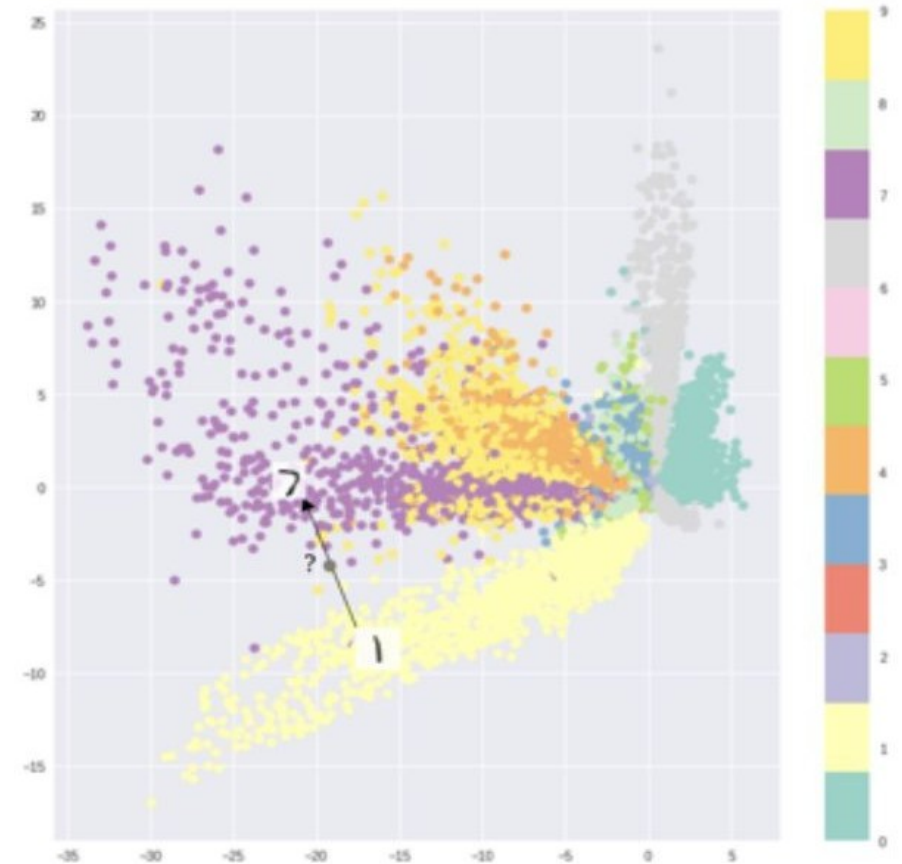


Sampling from autoencoder

Problem: Why can't we use normal auto-encoders ?

- If we choose an arbitrary latent vector, we may not be close to any points in the training set and the reconstruction is garbage!
- We need to ensure that the latent space has a proper structure and maps valid output

Latent space on MNIST





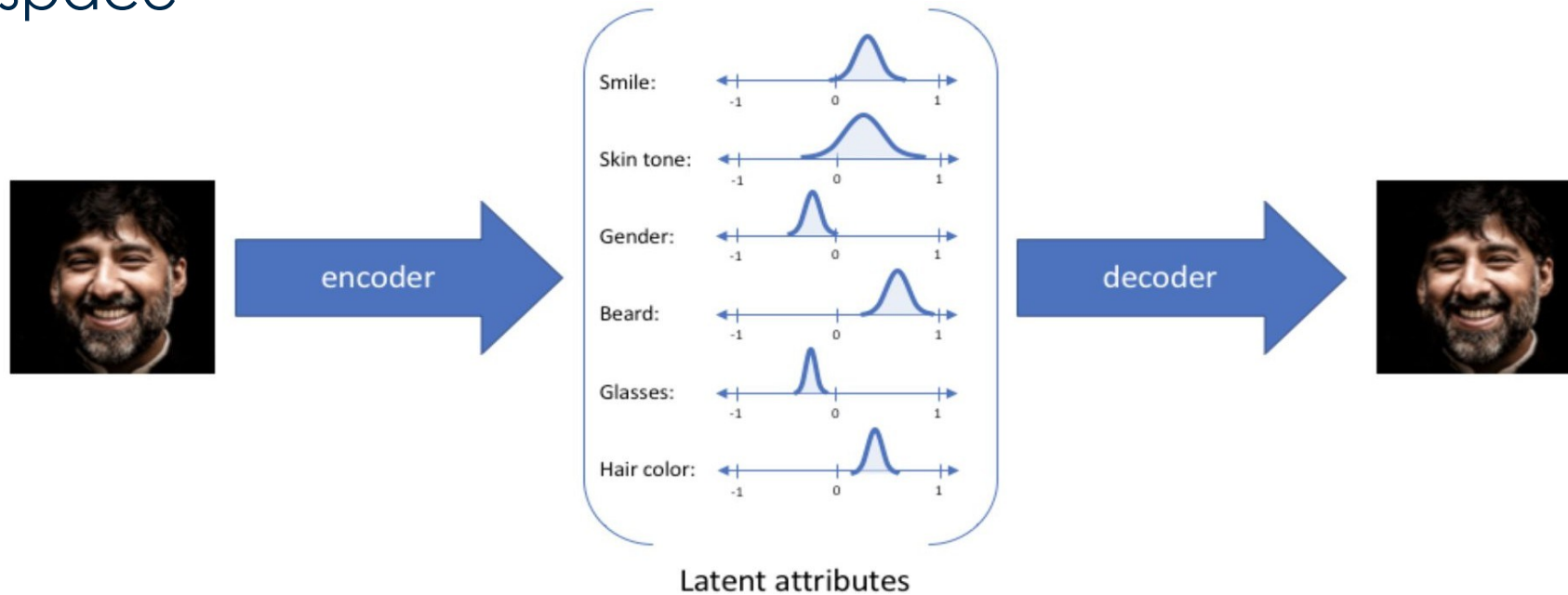
Politecnico
di Torino

From autoencoders to variational
autoencoders



The key ingredient

- Map the complicated data distribution to a simpler distribution (encoder) we can sample from
- Distributions rather than deterministic values ensure that close points in latent space lead to similar reconstructions – giving “meaning” to the latent space



Variational autoencoders

Our goal is to learn the data distribution

$$p_{\phi}(x) = \int \underbrace{p_{\phi}(x|z)}_{\text{parametric function}} \underbrace{p(z)}_{\text{latent variables}} dz$$

parametric function → encoder

latent variables → prior

$$p(z) = \mathcal{N}(0, I)$$

The integral over all latent codes is **intractable** for non-linear decoders
→ need a variational approach

Variational autoencoders

We introduce the variational distribution

$$q_{\theta}(z|x) \approx p_{\phi}(z|x) = \frac{p_{\phi}(x|z)p(z)}{p(x)}$$

parametric function → decoder

Then, rewrite the formula $p_{\phi}(x) = \int p_{\phi}(x|z)p(z)dz = \int p_{\phi}(x, z) dz$

$$\log p_{\phi}(x) = \log \int q_{\theta}(z|x) \frac{p_{\phi}(x, z)}{q_{\theta}(z|x)} dz = \log \mathbb{E}_{z \sim q_{\theta}(z|x)} \left[\frac{p_{\phi}(x, z)}{q_{\theta}(z|x)} \right]$$

Variational autoencoders

Apply Jensen inequality

$$\log p_{\phi}(x) \geq \mathbb{E}_{z \sim q_{\theta}(z|x)} \left[\log \frac{p_{\phi}(x, z)}{q_{\theta}(z | x)} \right]$$

and rewrite as a sum of individual terms:

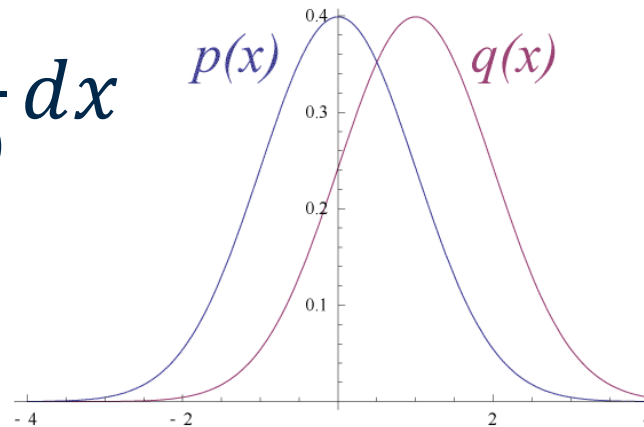
$$\begin{aligned} \log p_{\phi}(x, z) &= \log p_{\phi}(x | z) + \log p(z) \\ \mathbb{E}_q \left[\log \frac{p_{\phi}(x, z)}{q_{\theta}(z | x)} \right] &= \mathbb{E}_q [\log p_{\phi}(x | z)] + \mathbb{E}_q [\log p(z) - \log q_{\theta}(z | x)] \end{aligned}$$

Reinterpreting the loss as KL divergence

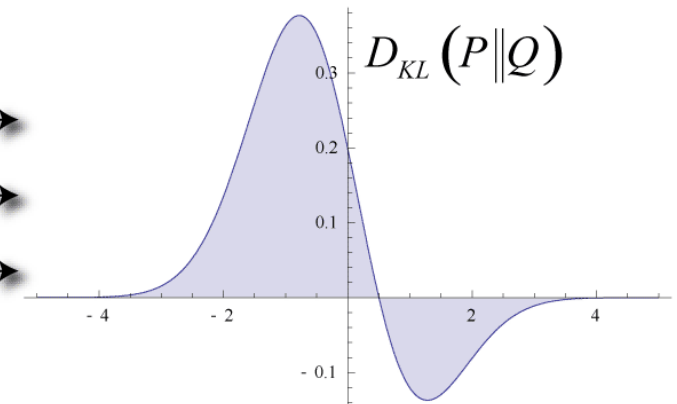
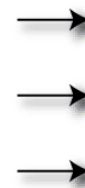
$$\mathbb{E}_q[\log p(z) - \log q_\theta(z | x)] = -\mathbb{E}_q \left[\log \frac{q_\theta(z | x)}{p(z)} \right] = -D_{\text{KL}}(q_\theta(z | x) \parallel p(z))$$

$$KL(P \parallel Q) = \int P(x) \log \frac{P(x)}{Q(x)} dx$$

Kullback-Leiber Divergence measures how one probability distribution P differs from a second, reference probability distribution $Q \rightarrow$ not a true distance function, as it is not symmetric



Original Gaussian PDF's



KL Area to be Integrated

Maximizing the ELBO (Evidence Lower BOund)

$$\log p_{\phi}(x) \geq \mathcal{L}_{\text{ELBO}}(\theta, \phi; x) = \underbrace{\mathbb{E}_{z \sim q_{\theta}(z|x)} [\log p_{\phi}(x | z)]}_{\text{Reconstruction}} - \underbrace{D_{\text{KL}}(q_{\theta}(z | x) \parallel p(z))}_{\text{Latent regularisation}}$$

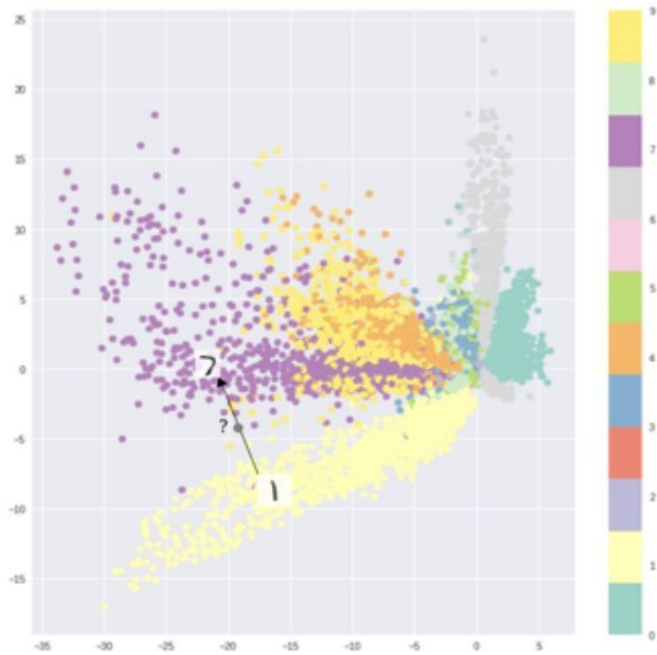
Reconstruction term: encourages the decoder to faithfully recover x from a sampled latent z

Latent regularization:

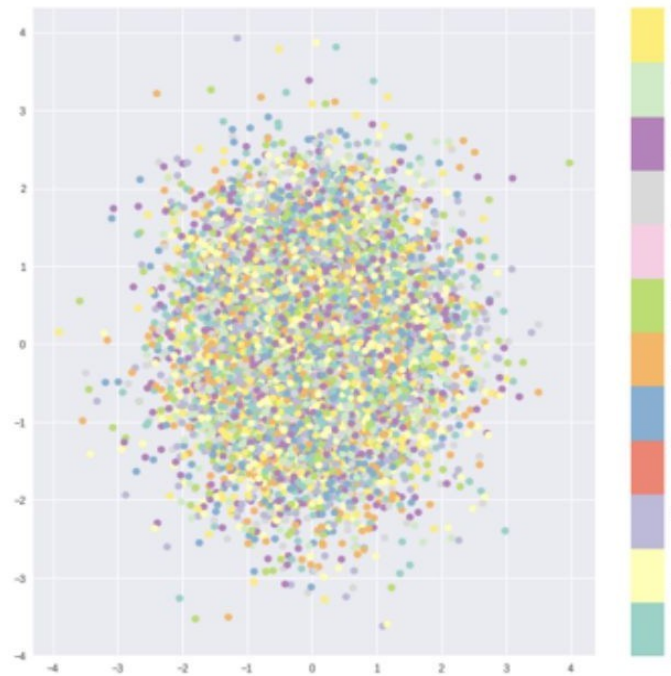
- penalises the encoder for straying from the standard Gaussian prior
- shapes the latent space into a smooth, sampleable manifold.

Learning the distribution of hand-written digits

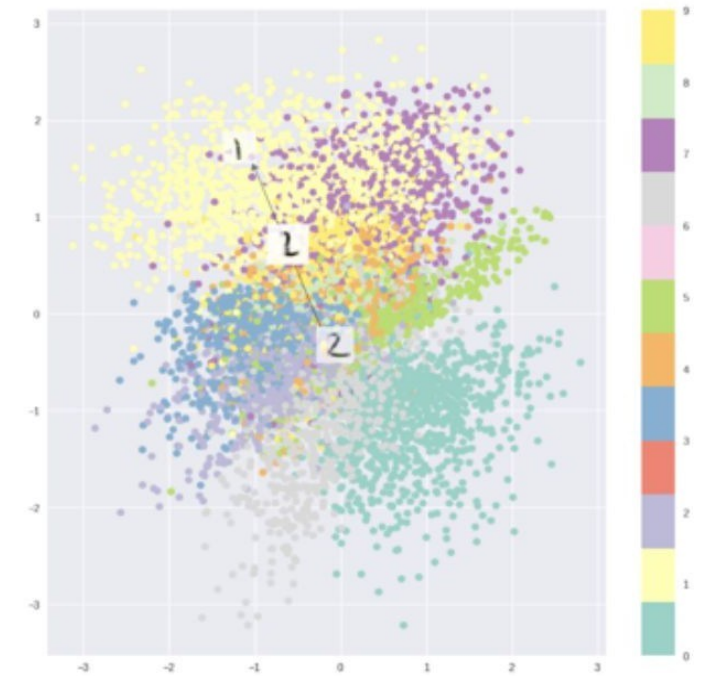
Only reconstruction loss



Only KL divergence

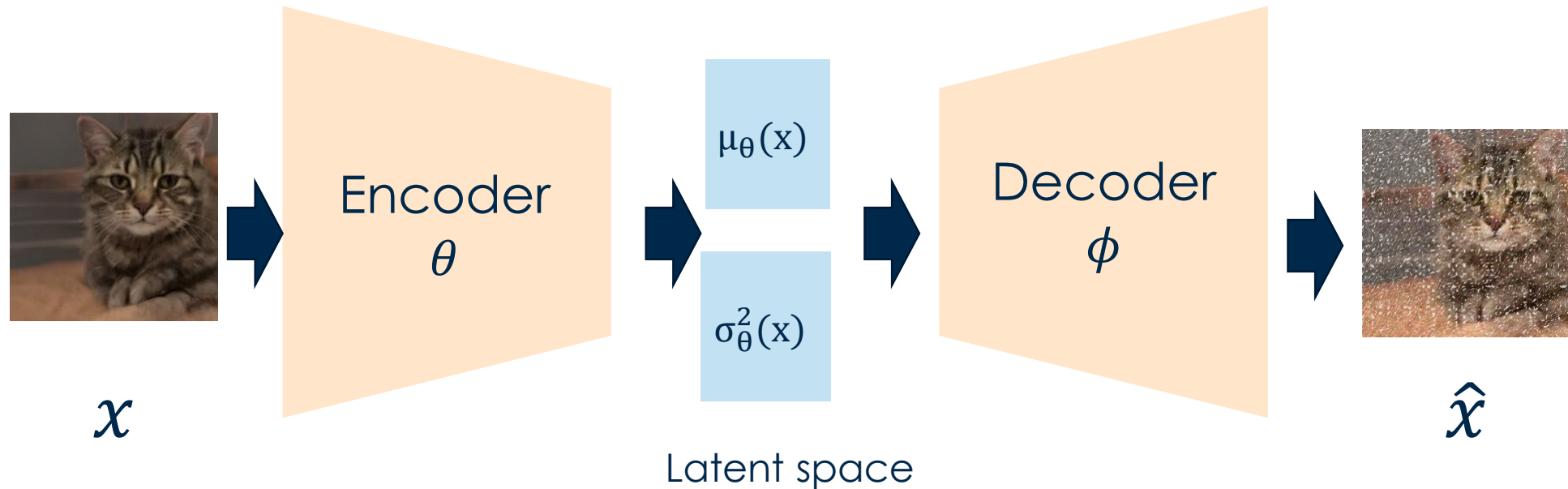


Combination



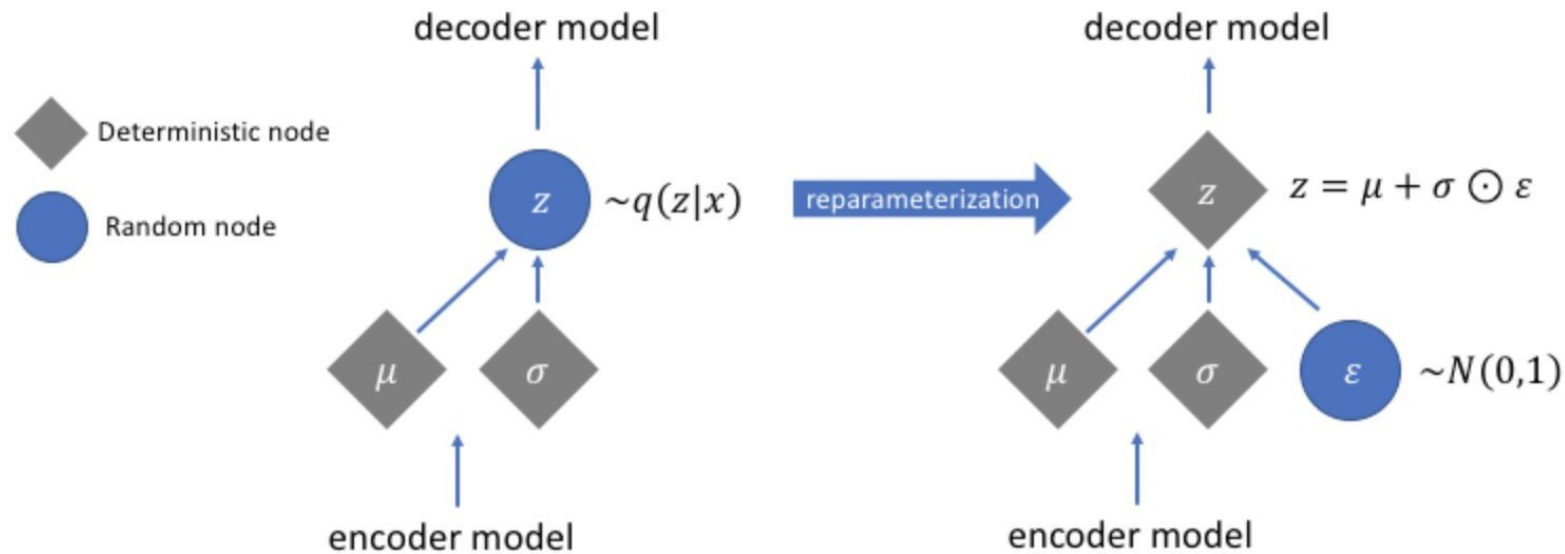
Gaussian decoder

- Uses a Gaussian decoder
- Encoder parametrizes a Gaussian distribution $q_{\theta}(z|x) = \mathcal{N}(z; \mu_{\theta}(x), \text{diag}(\sigma_{\theta}^2(x)))$
- It can be shown that the reconstruction error reduces to a mean-squared error loss



The re-parametrization trick

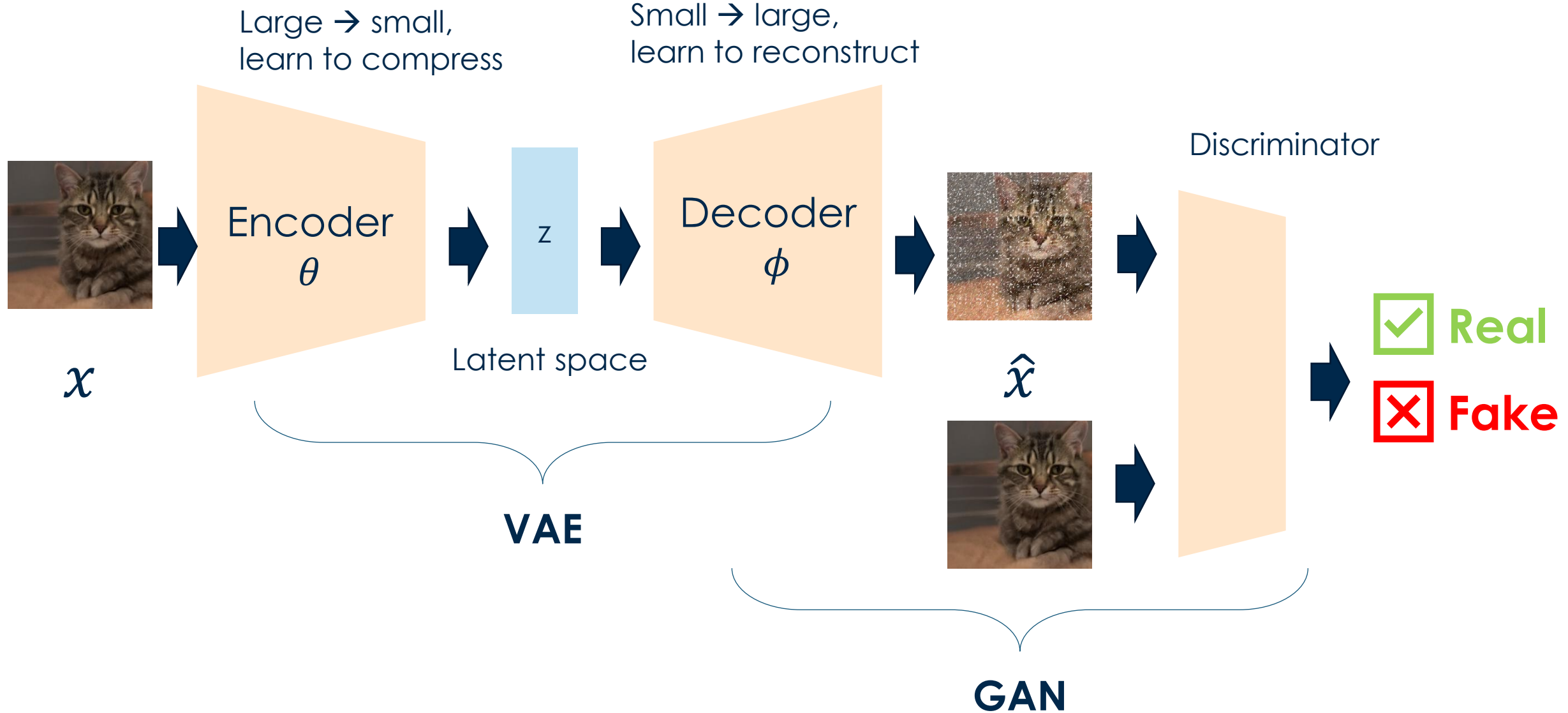
- Sampling is not a deterministic/differentiable process: we cannot backpropagate
- Solution: put the random process outside the network by adding noise



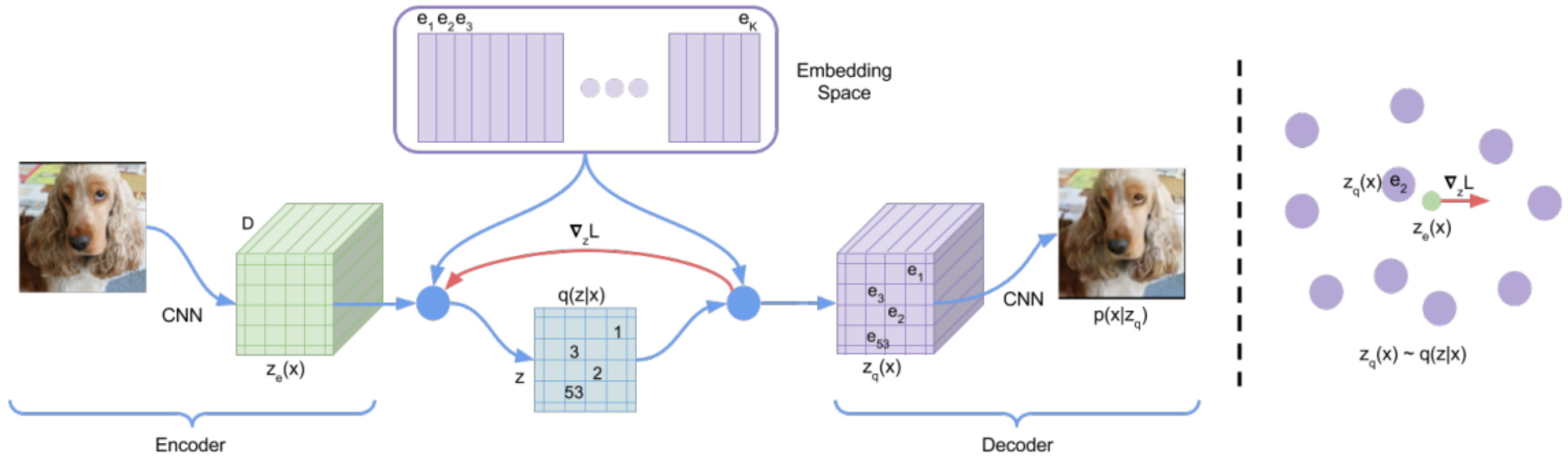
Issues with variational auto-encoders

- The decoder must output a single mean μ (z)
 - The output of VAEs is **blurry**
 - Cannot be fixed by making the networks deeper
 - it is a fundamental consequence of the Gaussian decoder objective, rooted in the least-squares nature of the reconstruction loss
- Latent space **collapse**
 - The decoder is often "too powerful," making it easier to generate output without using latent codes.
 - Latent space becomes indistinguishable from the prior, causing independent and identically distributed samples
- Possible solutions
 - Hierarchical VAEs
 - Vector quantized VAEs

VAE-GAN



Vector quantized VAE



<https://arxiv.org/pdf/1711.00937>

Major generative model families

Autoregressive Models

GPT, PixelCNN, WaveNet

- Computes exact likelihood
- + Multi-modal, flexible model
- Slow inference (token-by-token)
- Depends on tokenization

Variational Autoencoders

VAE, β -VAE, VQVAE

- Explicit latent space modelling
- May serve as building blocks for other models
- + Interpretability of the feature space
- + Fast inference
- + Effective representation learning
- Blurry images, low realism

GANs

GAN, StyleGAN, BigGAN

- Implicit distribution
- + Fast inference (single step)
- + Photo-realism
- Unstable training

Normalizing Flows

Glow, RealNVP, MAF

Exact inference

Diffusion Models

DDPM, Stable Diffusion, DALL-E

State of the art

Energy-Based Models

EBM, NCSN, ScoreNet

Flexible density